

Dry running codes
Stack :-

1) StackArrayIntDemo :-

Capacity = 3

1 → Push → 10

1 → Push → 20

1 → Push → 30

2 → Pop

3 → Peek

4 → Display

5 → Exit

Start :- top = -1

Stack = [, ,]

Push 10

top = 0

Stack = [10, ,]

Push 20

→ top = 1

Stack = [10, 20,]

Push 30

top = 2

Stack = [10, 20, 30]

Pop

30 is removed

top = 1

Peek

top element = 20

top = 1

Display

20 10

Exit

Operation	Stack	Top	O/P
		0	10 pushed
Push 10	[10, -, -]	1	20 pushed
Push 20	[10, 20, -]	2	30 pushed
Push 30	[10, 20, 30]	1	30 popped
Pop	[10, 20, 30]	1	Top = 20
Peek	[10, 20, 30]	1	20 10
display	[10, 20, 30]		

2) Stack Demo $\rightarrow$ Parentheses Matching

I'm taking ~~Stack~~ this expression

Expression = $\{ [()] \}$

Read	Operation	Stack	Top
{	Push	{	0
[	Push	{[	1
(	Push Pop (match)	{[(	2
)	Pop (match)	{[	1
]	Pop (match)	{	0
}	Pop (match)	empty	-1

Result = Balanced

4) Reverse String Stack Demo

Suppose $\Rightarrow$ HELLO
input

Step 1 - Push.

H $\rightarrow$ [H]

E $\rightarrow$ [H, E]

L $\rightarrow$ [H, E, L]

L $\rightarrow$ [H, E, L, L]

O $\rightarrow$ [H, E, L, L, O]

stack $\Rightarrow$ Top $\Rightarrow$

O

L

L

E

H

Pop $\Rightarrow$ O

L

L

E

H

O/P $\Rightarrow$ OLLEH

5) Expression Evaluation Demo

Token	Action	Stack
2	Push.	[2]
3	Push.	[2, 3]
1	Push.	[2, 3, 1]
*	$3 \times 1 = 3$	[2, 3]
+	$2 + 3 = 5$	[5]
9	Push.	[5, 9]
-	$5 - 9 = -4$	[-4]

* Queue

FIFO = First In, First Out

1) QueueDemo.java. $\Rightarrow$
Initial $\rightarrow$

front = -1, rear = -1

enqueue(10)

front = 0, rear = 0
[10, -, -, -, -]

enqueue(20)

front = 0, rear = 1
[10, 20, -, -, -]

enqueue(30)

front = 0, rear = 2
[10, 20, 30, -, -]

dequeue()
10 removed.
front = 1

enqueue(40)

rear = 3

[10, 20, 30, 40, -]

display()

→ 20 30 40

2) Circular Queue Demo :-

enqueue(10) → F = 0 R = 0

enqueue(20) → F = 0 R = 1

enqueue(30) → F = 0 R = 2

enqueue(40) → F = 0 R = 3

dequeue() → 10 removed → F = 1

dequeue() → 20 removed → F = 2

enqueue(50) → R = 4

enqueue(60) → R = 0 ← Circular

queue = 30 40 50 60